

Implementation Report

GitHub Repositories Node

MachinaOS / OpenCompany

Soumik Roy

July 13, 2026

Contents

1	Introduction	2
2	Objectives	2
3	Understanding the Plugin System	2
4	Implementation Process	3
4.1	Step 1: Creating the Plugin Directory	3
4.2	Step 2: Creating Required Files	3
5	Node Architecture	3
5.1	Input Parameters	4
5.2	Output Model	4
6	GitHub API Integration	4
7	Workflow Registration	5
8	Testing	5
8.1	Plugin Loading	5
8.2	Authentication	5
8.3	API Communication	5
8.4	Output Validation	5
9	Project Structure After Implementation	6
10	Result	6
11	Challenges Faced	6
12	Future Improvements	7
13	Conclusion	7

1 Introduction

MachinaOS follows a modular plugin-based architecture where every capability is implemented as an independent workflow node. These nodes can be connected inside the visual workflow editor and executed by the workflow engine.

The objective of this task was to extend the existing GitHub integration by creating a new node named **GitHub Repositories**. The purpose of this node is to authenticate using a GitHub Personal Access Token and retrieve all repositories belonging to the authenticated GitHub account.

The node was successfully integrated into the workflow system and appeared as a new draggable node inside the workflow editor. Upon execution, it communicates with the GitHub REST API and returns repository information in JSON format.

2 Objectives

The major objectives of this implementation were:

- Understand the MachinaOS plugin architecture.
- Learn how workflow nodes are registered automatically.
- Create a new GitHub workflow node.
- Authenticate using GitHub Personal Access Tokens.
- Fetch repositories from GitHub REST API.
- Return structured JSON output to downstream workflow nodes.
- Successfully integrate the node into the visual workflow editor.

3 Understanding the Plugin System

Before implementation, the following internal documentation was studied:

- Plugin System Documentation
- Server Nodes Documentation

From these documents, the following workflow was understood:

1. Every node exists inside
`server/nodes/<category>/<node>/`
2. Every node contains an `__init__.py` file.
3. The node inherits from one of:
 - `BaseNode`
 - `ActionNode`

- TriggerNode
- ToolNode

4. Every node defines

- Input Parameters (Pydantic Model)
- Output Model
- Operation Method

5. Importing the node automatically registers it into MachinaOS.

This architecture allows developers to add new functionality without modifying the core workflow engine.

4 Implementation Process

The implementation was divided into several stages.

4.1 Step 1: Creating the Plugin Directory

A new folder was created inside the GitHub category.

```
server/
  nodes/
    github/
      github_repositories/
```

This folder contains all logic related to the GitHub Repository node.

4.2 Step 2: Creating Required Files

The following files were created.

File	Purpose
<i>init.py</i>	Main node implementation
meta.json	Node metadata
icon.svg	Node icon displayed inside workflow editor

No modifications were required to the existing GitHub credentials because the node reused the existing authentication mechanism.

5 Node Architecture

The implementation follows the ActionNode architecture.

The major components are:

5.1 Input Parameters

A Pydantic class was created to define the node input.

Since no user input was required, the parameter class remained empty.

5.2 Output Model

A structured output model was defined for returning repository data.

The output contains:

- Repository count
- Repository names
- Visibility
- Default branch
- Description
- Repository URL

6 GitHub API Integration

The node authenticates using a GitHub Personal Access Token stored inside the environment variables.

Authentication Header:

Authorization: Bearer <TOKEN>

The node sends a GET request to the GitHub REST API.

The response is parsed and transformed into structured JSON before being returned to the workflow engine.

Example returned information includes:

- Repository Name
- Full Repository Name
- Visibility
- Description
- HTML URL
- Default Branch
- Programming Language

7 Workflow Registration

One important aspect of MachinaOS is automatic plugin discovery.

After placing the implementation inside

```
server/nodes/github/github_repositories/
```

and restarting the server, the workflow engine automatically detected the new node.

No manual registration was required.

This demonstrates the modular design of the plugin system.

8 Testing

Testing consisted of several stages.

8.1 Plugin Loading

After restarting MachinaOS, the node appeared inside the GitHub category without errors.

8.2 Authentication

The Personal Access Token stored inside the environment variables was successfully loaded.

8.3 API Communication

The node successfully established communication with GitHub REST API.

8.4 Output Validation

The workflow execution produced a JSON response containing all repositories associated with the authenticated account.

Example output included:

- Total repository count
- Repository names
- Visibility
- Description
- Repository URLs
- Default branch

9 Project Structure After Implementation

After completing the implementation, the GitHub module contained the following structure:

```
server/  
  nodes/  
    github/  
      __init__.py  
      credentials.py  
      github_repositories/  
        __init__.py  
        icon.svg  
        meta.json  
      ...
```

This modular structure keeps every node isolated while allowing easy future extensions.

10 Result

The implementation was successful.

The final workflow editor displayed the newly created GitHub Repositories node. When executed:

- Authentication succeeded.
- GitHub API request completed successfully.
- Repository list was fetched.
- Repository information was returned as structured JSON.
- The node became reusable inside any workflow.

The implementation demonstrated that custom workflow nodes can be added with minimal modifications to the existing system while leveraging the reusable credential infrastructure already available within MachinaOS.

11 Challenges Faced

Some challenges encountered during implementation were:

- Understanding the plugin registration mechanism.
- Understanding ActionNode inheritance.
- Learning the folder structure expected by MachinaOS.
- Correctly defining Pydantic input and output models.

- Configuring GitHub authentication.
- Debugging API response formatting.
- Ensuring the node appeared correctly inside the visual workflow editor.

12 Future Improvements

Several enhancements can be added to this node:

- Repository filtering.
- Pagination support.
- Search repositories by keyword.
- Fetch branches.
- Fetch contributors.
- Fetch issues.
- Fetch pull requests.
- Fetch commits.
- Support GitHub Organizations.
- Repository statistics.

13 Conclusion

The GitHub Repositories node was successfully implemented and integrated into MachinaOS using its plugin-based architecture. The implementation required understanding the internal node registration system, creating the appropriate directory structure, defining node metadata, implementing an ActionNode, and interacting with the GitHub REST API using an existing credential mechanism.

The final implementation demonstrates how new workflow capabilities can be added in a modular and reusable manner. The node is now available inside the workflow editor and can be incorporated into larger automation workflows that require GitHub repository information.

Here's the forked repo link: <https://github.com/soumik-sr/MachinaOS>